SIKSHA 'O' ANUSANDHAN
DEEMED TO BE UNIVERSITY

Admission Batch: **Session:**

MINOR PROJECT
FULL-STACK WEB DEVELOPMENT WITH MERN
[CSE 3839]

Submitted by

Name: Koushik Das

Registration No.: 2341004117

Branch: COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

Semester: 6th Section: 23413B2

Department of Computer Science & Information Technology

(ITER)

Jagamohan Nagar, Jagamara, Bhubaneswar, Odisha - 751030

DECLARATION

I hereby declare that the minor project titled “Library Managenment System” submitted by me is an original work carried out for the course “FULL-STACK WEB DEVELOPMENT WITH MERN”. All the information and data presented in this project are true to the best of my knowledge and belief .

Name of the Student: Koushik Das
Registration No: 2341004117
Course & Semester: ____BTECH (6TH)____

Signature of the Student: _____
Date: 12th MAY 2026

CONTENT

1. INTRODUCTION

The Library Management System is a Full Stack MERN web application developed to digitally manage and organize library records in a simple and efficient way. The project was built using MongoDB, Express.js, React.js, and Node.js with the aim of understanding practical full-stack development concepts along with database integration, API handling, and frontend-backend communication.

Traditional library systems often depend on manual registers and paperwork for storing information related to books, which becomes difficult to manage when the number of records increases. Searching, updating, or maintaining book details manually consumes time and can also lead to mistakes or duplicate records. This project attempts to solve those issues by creating a centralized digital system where books can be added, viewed, requested, and managed dynamically.

The application provides a responsive React frontend connected to a Node.js and Express backend through REST APIs. MongoDB is used as the database to store book information and request data. One of the important features implemented in this project is PDF upload functionality using Multer middleware, allowing digital book documents to be stored and accessed directly from the system.

During the development process, several real-world implementation challenges were faced such as API connection errors, frontend-backend integration issues, Axios request handling, React state management problems, and Multer file upload configuration. These debugging experiences helped in understanding how different technologies interact inside a complete MERN application.

The project demonstrates practical implementation of:

*** CRUD Operations**
*** REST API Development**
*** MongoDB Database Integration**
*** File Upload Handling**
*** React Router Navigation**
*** Dynamic Frontend Rendering**
*** Aggregation and Request Count Logic**

Overall, the project provides hands-on exposure to modern web application development and demonstrates how MERN stack technologies can be integrated together to create scalable and user-friendly systems.

2. OBJECTIVE OF WORK

The main objective of this project is to develop a digital Library Management System capable of handling major library operations efficiently using MERN stack technologies.

The project focuses not only on implementing features but also on understanding the complete workflow of frontend-backend communication, API integration, database handling, and file upload management in real-world web development.

The specific objectives of the project are:

*** To develop a complete Full Stack web application using MongoDB, Express.js, React.js, and Node.js.**
*** To create a responsive and user-friendly interface for managing books digitally.**
*** To implement CRUD operations for adding, viewing, updating, and deleting books.**
*** To reduce manual record maintenance and improve data organization.**
*** To integrate MongoDB database with backend APIs for dynamic data storage and retrieval.**
*** To implement PDF upload functionality using Multer middleware.**
*** To understand REST API architecture and frontend-backend communication using Axios.**
*** To implement request count logic for tracking frequently requested books.**
*** To improve practical knowledge of React state management, routing, and component-based architecture.**
*** To gain hands-on experience in debugging real-world development issues such as network errors, API failures, and file upload handling.**
*** To understand project structuring and modular coding practices used in full-stack development.**

The project also aims to strengthen practical understanding of modern web technologies and software development workflows used in industry-level applications.

3. Software and Hardware Requirement

1. Software Requirements
Operating System: Windows 10 or Windows 11
Frontend Stack: HTML5, CSS3, JavaScript (ES6+), and React.js for building the user interface
Backend Stack: Node.js runtime environment with the Express.js framework
Database Management: MongoDB (NoSQL) used with the Mongoose ODM
Development Tools: Visual Studio Code (IDE) and npm (Node Package Manager)
Browsers: Google Chrome or Microsoft Edge (for debugging and testing)
Middleware/Utilities: Multer (for handling multipart/form-data and file uploads)

2. Hardware Requirements
Processor: Intel Core i3 (or equivalent) and above
Memory (RAM): Minimum 4 GB (8 GB recommended for optimal performance with React and Chrome)
Storage: At least 20 GB of available disk space
Connectivity: Active Internet connection for package installation and database syncing
Peripherals: Standard Keyboard and Mouse/Trackpad

4. IMPLEMENTATION

The Library Management System was implemented using the MERN stack architecture where React.js handles the frontend user interface, Node.js and Express.js handle backend APIs, and MongoDB stores application data.

The frontend was designed using React components and React Router for navigation between pages such as Home, Books, Add Book, and Update Book pages. Axios was used to establish communication between the frontend and backend APIs.

The backend server was developed using Express.js and connected to MongoDB using Mongoose. Separate folders were maintained for routes, models, configuration files, and uploads to achieve modular project structure and maintainability.

During implementation, the following core functionalities were developed:

*** Adding books dynamically through forms**
*** Uploading PDF files using Multer middleware**
*** Fetching books from MongoDB database**
*** Deleting books dynamically**
*** Requesting books and increasing request count**
*** Aggregation-based filtering**
*** REST API communication between frontend and backend**

The project also involved debugging and resolving multiple real-world development issues such as:

*** Axios network errors**
*** Incorrect API endpoint handling**
*** CORS configuration issues**
*** Multer file upload mismatches**
*** React hook state management problems**
*** Frontend-backend synchronization errors**

Special attention was given to maintaining a clean folder structure and reusable component architecture. CSS styling was implemented separately using modular CSS files for better maintainability and responsive design.

The implementation process provided practical understanding of:

*** React component lifecycle**
*** Express middleware workflow**
*** MongoDB schema design**
*** File upload handling**
*** State management**
*** API testing and debugging**

The project successfully demonstrates how multiple technologies work together to create a complete Full Stack web application.

BACKEND SECTION

The backend of the Library Management System was developed using Node.js and Express.js to handle server-side operations and REST API communication. MongoDB was used as the database for storing information related to books and requests, while Mongoose was used for schema creation and database interaction.

The backend was structured in a modular way by separating configuration files, routes, models, and uploads into different folders for better maintainability and readability.

The backend implementation includes the following major functionalities:

Book Management APIs

REST APIs were implemented for:

*** Adding books**
*** Fetching all books**
*** Updating existing books**
*** Deleting books**
*** Requesting books dynamically**

These APIs allow smooth communication between the React frontend and MongoDB database.

MongoDB Integration

MongoDB was connected using Mongoose, which simplified database operations such as:

*** Creating book records**
*** Updating request count**
*** Retrieving all books**
*** Deleting records dynamically**

Separate schemas were created for:

*** Book collection**
*** Request collection**

PDF Upload Functionality

Multer middleware was implemented to handle PDF uploads. Uploaded PDF files are stored inside the uploads folder, while the filename is stored in MongoDB for future access.

During implementation, issues related to file upload handling and field matching between frontend and backend were debugged and resolved successfully.

Request Count Logic

A request-based logic was implemented where:

*** If a requested book already exists, its count value increases.**
*** If the book does not exist, a new record is automatically created.**

This feature helped in understanding conditional database operations and dynamic data management.

Aggregation Filtering

MongoDB aggregation pipeline was used for filtering books based on request count ranges. This provided practical exposure to advanced MongoDB query handling.

Backend Technologies Used

*** Node.js**
*** Express.js**
*** MongoDB**
*** Mongoose**
*** Multer**
*** CORS**

Backend Features

*** REST API Architecture**
*** CRUD Operations**
*** MongoDB Integration**
*** PDF Upload Support**
*** Request Count Increment Logic**
*** Aggregation Filtering**
*** Express Routing**
*** File Upload Handling**
*** API Error Handling**

The backend implementation provided practical understanding of server-side development, API creation, database connectivity, middleware usage, and debugging of real-world integration issues.

SOURCE CODE

Project Name : Library-management

CONFIG

Multer.js

```
const multer = require("multer");
const storage = multer.diskStorage({
   destination: function(req,file,cb){
      cb(null,"uploads/");
   },
   filename: function(req,file,cb){
      cb(null, Date.now() + "-" + file.originalname);
   }
```

```
});
const FileFilter = (req,file,cb) => {
   if(file.mimetype === "application/pdf"){
      cb(null,true);
   }else{
      cb(new Error("Only PDF files are allowed"),false);
   }
};
const upload = multer({
   storage: storage
});
module.exports = upload;
```

Models

Book.js

```
const mongoose = require("mongoose");
const bookSchema = new  mongoose.Schema({
   title:{type: String,required: true},
   author:{type:String,required: true},
   pdf:{type:String},
   count:{type:Number,default:1}
});
module.exports = mongoose.model("Book",bookSchema);
```

Request.js

```
const mongoose = require("mongoose");
const requestSchema = new mongoose.Schema({
   bookTitle:{type: String,required: true},
   requestedBy:{type: String,required: true},
   date:{type: Date,default: Date.now}
});
module.exports = mongoose.model("Request",requestSchema);
```

Routes

bookRoutes.js

```
const express = require("express");
const router = express.Router();
const Book = require("../models/Book");
const Request = require("../models/Request");
const upload = require("../config/multer");
const { log } = require("node:console");

router.post("/", upload.single("pdf"), async (req, res) => {
   try {
      console.log(req.body);
      console.log(req.file);
```

```
        const {title,author} = req.body;
        let book = await Book.findOne({ title: title });

        if (book) {
            book.count += 1;
            await book.save();
            return res.json({
                message: "Book already exists. Count increased. ",
                book
            });
        }

        const newBook = new Book({
            title,
            author,
            pdf: req.file ? req.file.filename : "",
            count: 1
        });
        await newBook.save();
        res.json({
            message: "New book added successfully",
            newBook
        });
    } catch (error) {
        console.log(error);
        res.status(500).json({
            message: "Server Error"
        });
    }
});
router.get("/", async (req, res) => {
    try {
        const books = await Book.find();
        res.json(books);
    } catch (error) {
        res.status(500).json({
            message: "Error fetching books"
        });

    }
});

router.put("/:id", async (req, res) => {
    try {
        const updatedBook = await Book.findByIdAndUpdate(
            req.params.id,
            req.body,
            { new: true }
        );
        res.json(updatedBook);
    } catch (error) {
        res.status(500).json({
            message: "Update failed"
        });
    }
```

```
});

router.delete("/:id", async (req, res) => {
  try {
    await Book.findByIdAndDelete(req.params.id);
    res.json({
      message: "Book deleted"
    });
  } catch (error) {
    res.status(500).json({
      message: "Delete failed"
    });
  }
});

router.post("/request", async (req, res) => {
  try {
    const { title, user } = req.body;
    let book = await Book.findOne({ title });

    if (book) {
      book.count += 1;
      await book.save();
    }

    else {
      book = new Book({
        title,
        author: "Unknown",
        count: 1
      });
      await book.save();
    }

    const request = new Request({
      bookTitle: title,
      requestedBy: user
    });
    await request.save();
    res.json({
      message: "Book requested successfully"
    });
  } catch (error) {
    res.status(500).json({
      message: "Request failed"
    });
  }
});

router.get("/filter", async (req, res) => {
  try {
    const min = parseInt(req.query.min);
    const max = parseInt(req.query.max);
    const books = await Book.aggregate([
      {
```

```
            $match: {
              count: {
                $gte: min,
                $lte: max
              }
            }
          }
        ]);
        res.json(books);
    } catch (error) {
        res.status(500).json({
          message: "Filter failed"
        });
    }
});
module.exports = router;
```

Server.js

```
require("dotenv").config();
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
const bookRoutes = require("./routes/bookRoutes");
const app = express();
app.use(cors());
app.use(express.json());
app.use("/uploads", express.static("uploads"));
mongoose.connect(process.env.MONGO_URI)
  .then(() => {
    console.log("MongoDB Connected");
  })
  .catch((error) => {
    console.log("MongoDB Connection Error:", error);
  });
app.use("/api/books", bookRoutes);
const PORT = 5000;
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
})
```

FRONTEND SECTION

IMPLEMENTATION

The frontend of the Library Management System was developed using React.js to create an interactive and responsive user interface. React Router was used for navigation between different pages, while Axios was used for API communication with the backend server.

The frontend was organized using reusable components, pages, and separate CSS files to maintain clean structure and improve code readability.

The frontend implementation includes the following major functionalities:

Responsive User Interface

A responsive layout was designed using CSS to ensure proper alignment and usability of the application on different screen sizes.

Separate styling files were created for:

* Navbar
* Forms
* Book Cards
* Global Application Styling

React Router Navigation

React Router DOM was implemented to manage navigation between pages such as:

* Home Page
* Add Book Page
* Books Page
* Update Book Page

This allowed smooth page transitions without refreshing the application.

Add Book Functionality

A form-based interface was developed for adding books dynamically. Users can:

* Enter book title
* Enter author name
* Upload PDF files

React state management using useState hooks was implemented for handling form data.

Dynamic Book Display

Books stored in MongoDB are fetched through backend APIs and displayed dynamically using reusable React components.

Each book card displays:

* Book title
* Author name

* Request count
* PDF access link
* Delete and Request buttons

API Integration

Axios was used to send HTTP requests between frontend and backend for:

* Fetching books
* Adding books
* Requesting books
* Deleting books

During implementation, several frontend-backend communication issues such as Axios network errors and API routing mismatches were debugged and resolved.

Frontend Technologies Used

* React.js
* React Router DOM
* Axios
* CSS3
* JavaScript

Frontend Features

* Responsive UI
* React Router Navigation
* Add Book Form
* Dynamic Book Display
* PDF Upload Support
* Request Book Functionality
* Delete Book Functionality
* Modular Component Structure
* API Integration using Axios
* State Management using React Hooks

The frontend implementation helped in understanding component-based architecture, routing, state handling, API integration, and responsive web design using React.js.

Components

AddBookForm.js

```
import React, { useState,useRef } from "react";
import api from "../api";
import "../styles/Form.css";
function AddBookForm() {
   const [title, setTitle] = useState("");
   const [author, setAuthor] = useState("");
   const [pdf, setPdf] = useState(null);
   const fileInputRef = useRef(null);
   const handleSubmit = async (e) => {
   e.preventDefault();
   const formData = new FormData();
   formData.append("title", title);
   formData.append("author", author);
   formData.append("pdf", pdf);
   try {
      await api.post("/", formData);
      alert("Book Added Successfully");
      setTitle("");
      setAuthor("");
      setPdf(null);
      fileInputRef.current.value = "";
   }
   catch(error) {
      console.log(error);
   }
};
return (
   <div className="form-container">
      <h2>Add Book</h2>
    <form onSubmit={handleSubmit}>
        <input
        type="text"
        placeholder="Book Title"
        value={title}
        onChange={(e) => setTitle(e.target.value)}
        required
       />
       <input
        type="text"
        placeholder="Author Name"
        value={author}
        onChange={(e) => setAuthor(e.target.value)}
        required
       />
       <input
        type="file"
        accept=".pdf"
        ref={fileInputRef}
        onChange={(e) => setPdf(e.target.files[0])}
       />
```

```
      <button type="submit">
        Add Book
      </button>
    </form>
  </div>
 );
}
export default AddBookForm;
```

BookCard.js

```
import React from "react";
import "../styles/BookCard.css";
function BookCard({ book, onDelete, onRequest }) {
 return (
  <div className="book-card">
   <h3>{book.title}</h3>
   <p>{book.author}</p>
   <p>Count: {book.count}</p>
   {
    book.pdf &&
    <a
     href={`http://localhost:5000/uploads/${book.pdf}`}
     target="_blank"
     rel="noreferrer"
    >
     Open PDF
    </a>
   }
   <div className="btn-group">
    <button onClick={() => onRequest(book.title)}>
     Request
    </button>
    <button onClick={() => onDelete(book._id)}>
     Delete
    </button>
   </div>
  </div>
 );
}
export default BookCard;
```

Navbar.js

```
import React from "react";
import { Link } from "react-router-dom";
import "../styles/Navbar.css";


function Navbar() {
  return (
  <nav className="navbar">
    <h2>Library Management</h2>
    <div className="nav-links">
     <Link to="/">Home</Link>
```

```
        <Link to="/books">Books</Link>
        <Link to="/add-book">Add Book</Link>
       </div>
    </nav>
    );
}
export default Navbar;
```

Pages

AddBookPage.js

```
import React from "react";
import AddBookForm from "../components/AddBookForm";
function AddBookPage() {
    return (
     <div>
       <AddBookForm />
     </div>
    );
}
export default AddBookPage;
```

BookPage.js

```
import React, { useEffect,useState } from "react";
import api from "../api";
import BookCard from "../components/BookCard";
function BooksPage() {
  const [books, setBooks] = useState([]);
  const [error, setError] = useState("");
  const fetchBooks = async () => {
    try {
      const response = await api.get("/");
      setBooks(response.data);
    }
    catch(error) {
      console.log(error);
      setError("Backend connection failed");
    }
  };
  useEffect(() => {
    fetchBooks();
  }, []);
  const deleteBook = async (id) => {
    try {
      await api.delete(`/${id}`);
      fetchBooks();
    }
    catch(error) {
      console.log(error);
    }
  };
```

```
  const requestBook = async (title) => {
   try {
    await api.post("/request", {
     title,
     user: "Koushik"
    });
    fetchBooks();
   }
   catch(error) {
    console.log(error);
   }
  };
  return (
   <div>
    {
     error &&
      <h2>{error}</h2>
    }
    <div className="books-grid">
     {
      books.map((book) => (
       <BookCard
        key={book._id}
        book={book}
        onDelete={deleteBook}
        onRequest={requestBook}
       />
      ))
     }
    </div>
   </div>
  );
}
export default BooksPage;
```

Home.js

```
import React from "react";
function Home(){
 return (
  <div className="page">
   <h1>Koushik's Library Management System</h1>
   <p>
    Your Gateway to Organized Knowledge and Seamless Book Management
   </p>
  </div>
 );
}
export default Home;
```

UpdateBookPage.js

```
import React from "react";
function UpdateBookPage() {
  return (
    <div className="page">
      <h1>Update Book Feature</h1>
      <p>
        Update functionality can be implemented here.
      </p>
    </div>
  );
}
export default UpdateBookPage;
```

Styles

App.css

```
body {
  margin: 0;
  font-family: Arial, sans-serif;
  background: #f5f5f5;
}
.page {
  padding: 40px;
  text-align: center;
}
.books-grid {
  display: flex;
  flex-wrap: wrap;
  gap: 20px;
  padding: 20px;
  justify-content: center;
}
```

BookCard.css

```
.book-card {
  width: 280px;
  background: white;
  border-radius: 12px;
  padding: 15px;
  box-shadow: 0px 4px 10px rgba(0,0,0,0.2);
  text-align: center;
}
.book-card img {
  width: 100%;
  height: 250px;
  object-fit: cover;
  border-radius: 10px;
}
.btn-group {
  display: flex;
  justify-content: space-between;
  margin-top: 10px;
}
.btn-group button {
  padding: 8px 12px;
```

```
  border: none;
  background: #222;
  color: white;
  border-radius: 5px;
  cursor: pointer;
}
```

Form.css

```
.form-container {
  width: 400px;
  margin: 40px auto;
  background: white;
  padding: 30px;
  border-radius: 10px;
  box-shadow: 0px 4px 10px rgba(0,0,0,0.2);
}
.form-container form {
  display: flex;
  flex-direction: column;
  gap: 15px;
}
.form-container input {
  padding: 12px;
  border: 1px solid #ccc;
  border-radius: 5px;
}
.form-container button {
  padding: 12px;
  border: none;
  background: #222;
  color: white;
  cursor: pointer;
  border-radius: 5px;
}
```

Navbar.css

```
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 20px;
  background-color: #222;
  color: white;
}
.nav-links {
  display: flex;
  gap: 20px;
}
.nav-links a {
  color: white;
  text-decoration: none;
```

```
  font-weight: bold;
}
.nav-links a:hover {
  color: cyan;
}
```

Api.js

```
import { render, screen } from '@testing-library/react';
import App from './App';

test('renders learn react link', () => {
  render(<App />);
  const linkElement = screen.getByText(/learn react/i);
  expect(linkElement).toBeInTheDocument();
});
```

App.js

```
import React from "react";
import { BrowserRouter,Routes,Route } from "react-router-dom";
import Navbar from "./components/Navbar";
import Home from "./pages/Home";
import BooksPage from "./pages/BooksPage";
import AddBookPage from "./pages/AddBookPage";
import UpdateBookPage from "./pages/UpdateBookPage";
import "./styles/App.css";
function App() {
  return (
  <BrowserRouter>
  <Navbar />
    <Routes>
      <Route path="/" element={<Home />} />
       <Route path="/books" element={<BooksPage />} />
         <Route path="/add-book" element={<AddBookPage />} />
          <Route path="/update/:id" element={<UpdateBookPage />} />
    </Routes>
  </BrowserRouter>
  );
}
export default App;
```

Index.js

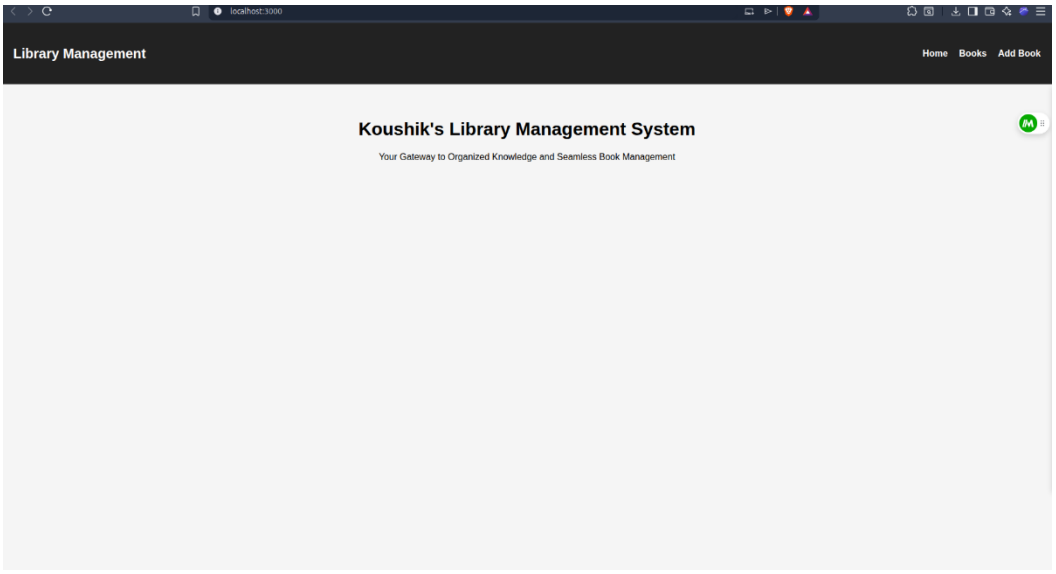
```
import React from 'react';
import ReactDOM from 'react-dom/client';
import './index.css';
import App from './App';
import reportWebVitals from './reportWebVitals';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
```

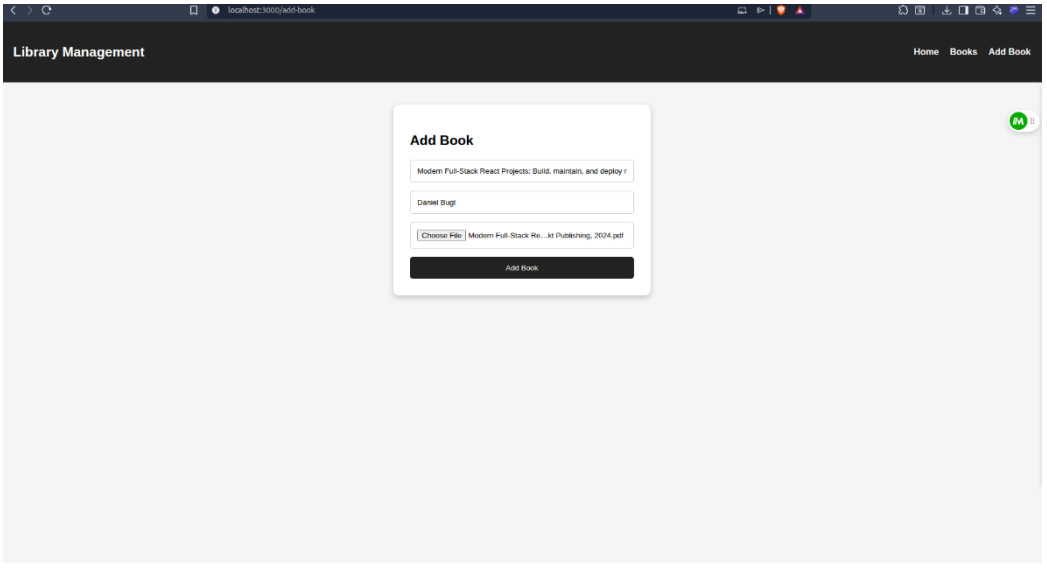
```
);
reportWebVitals();
```

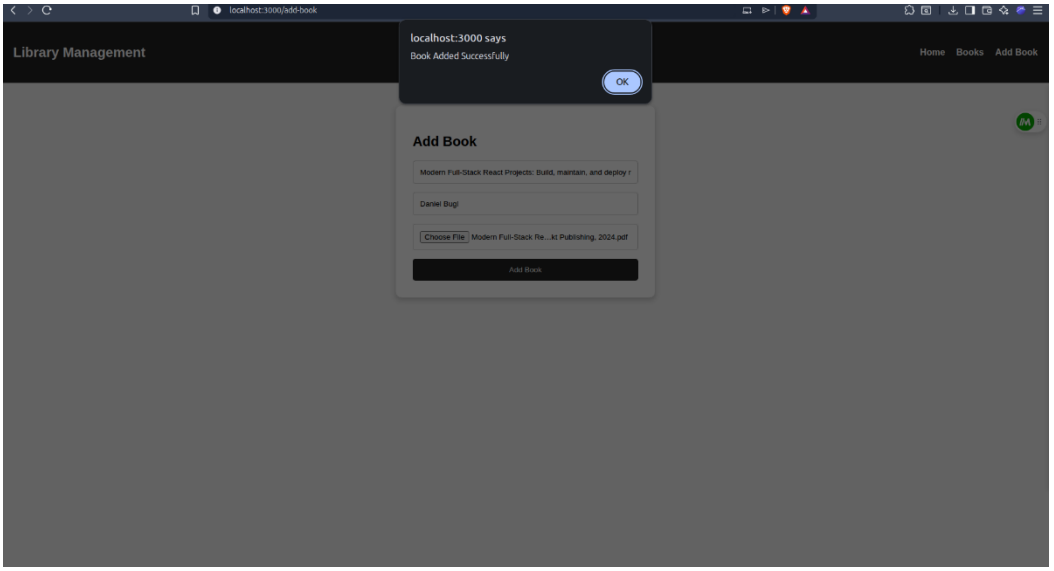
Index.css

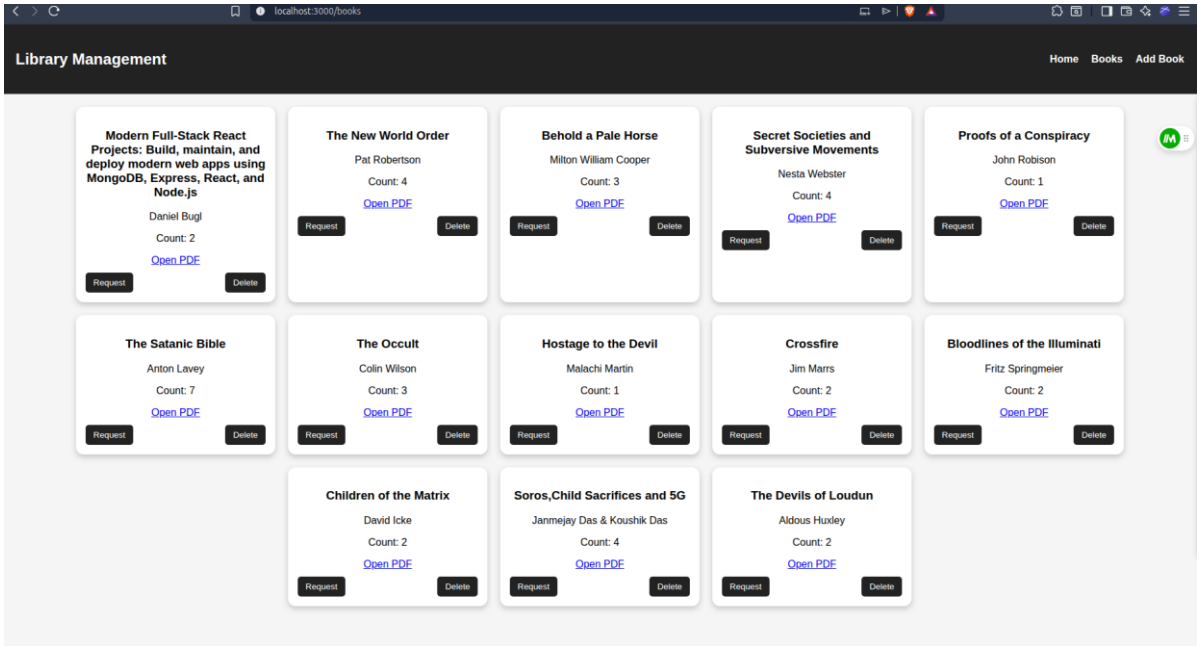
```
body {
  margin: 0;
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', 'Roboto', 'Oxygen',
    'Ubuntu', 'Cantarell', 'Fira Sans', 'Droid Sans', 'Helvetica Neue',
    sans-serif;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
}

code {
  font-family: source-code-pro, Menlo, Monaco, Consolas, 'Courier New',
    monospace;
}
```

Output


localhost:3000
Library Management
Home
Books
Add Book
Koushik's Library Management System
Your Gateway to Organized Knowledge and Seamless Book Management


localhost:3000/add-book
Library Management
Home
Books
Add Book
Add Book
Modern Full-Stack React Projects: Build, maintain, and deploy r
Daniel Bugl
Choose File
Modern Full-Stack Re...kt Publishing, 2024.pdf
Add Book


localhost:3000/add-book
Library Management
Home
Books
Add Book
localhost:3000 says
Book Added Successfully
OK
Add Book
Modern Full-Stack React Projects: Build, maintain, and deploy r
Daniel Bugl
Choose File
Modern Full-Stack Re...kt Publishing, 2024.pdf
Add Book



5. Results

The Library Management System was successfully developed and tested as a complete Full Stack MERN application. The project achieved the intended functionality of managing books digitally through a responsive frontend and a connected backend system using MongoDB.

During the implementation process, multiple practical concepts of full-stack development were explored and applied successfully. The system was able to:

*** Add books dynamically**
*** Upload and manage PDF files**
*** Display books from MongoDB**
*** Delete books**
*** Request books dynamically**
*** Increment request count automatically**
*** Handle frontend-backend API communication properly**

The React frontend successfully interacted with the Express backend using Axios API calls, while MongoDB stored and retrieved records dynamically without manual intervention. The project structure was organized into components, pages, routes, models, and configuration files to maintain modularity and readability.

One of the major achievements of the project was

implementing PDF upload functionality using Multer middleware. Initially, image uploads were considered, but later the system was improved to support PDF-based digital books for a more realistic library workflow. Several issues related to file uploads, field mismatches, API connection failures, and frontend-backend synchronization were encountered and resolved during development.

The debugging process itself became an important learning experience. Problems such as:

*** Axios network errors**
*** Multer configuration mismatches**
*** React hook issues**
*** API routing problems**
*** Backend connection failures**
*** File state reset handling**
were identified and fixed step-by-step during project implementation.

The request count logic worked successfully according to the project requirements:

*** New books were added with count = 1**
*** Existing books increased their request count dynamically**

The frontend interface was designed using reusable React components and CSS styling to provide a cleaner and more user-friendly experience. Navigation between pages using React Router was implemented successfully without page reloads.

Overall, the project provided practical exposure to:

*** MERN stack architecture**
*** REST API development**
*** MongoDB database handling**
*** React component-based design**
*** State management**
*** Middleware usage**
*** File upload handling**

*** Real-world debugging and integration workflow**

The final output demonstrated a working digital Library Management System capable of performing core library operations efficiently through a modern web-based interface.

6. Conclusions

The Library Management System was successfully completed as a Full Stack MERN web application capable of handling essential library operations digitally. The project fulfilled its primary objective of creating a user-friendly and efficient platform for managing books and maintaining records using modern web technologies.

The development of this project provided deep practical understanding of MongoDB, Express.js, React.js, and Node.js along with their integration into a single working application. Instead of only learning theoretical concepts, the project involved real implementation, debugging, testing, and frontend-backend synchronization.

A major learning outcome from this project was understanding how real-world development involves continuous debugging and problem-solving. Throughout the project, multiple challenges were faced related to:

*** Axios API communication**
*** React state management**
*** React Router navigation**
*** Multer PDF upload handling**
*** Backend routing**
*** MongoDB data storage**
*** Frontend-backend connection errors**

Resolving these issues helped in understanding the actual workflow of full-stack application development and improved practical coding confidence significantly.

The project also demonstrated the importance of:

*** modular folder structure**
*** reusable components**
*** API-based architecture**
*** middleware handling**
*** database schema design**
*** clean UI organization**

The final system successfully reduced manual record handling and provided a more organized digital solution for managing books. Features such as PDF uploads, request count tracking, CRUD operations, and dynamic rendering made the project more practical and realistic compared to a basic static application.

This project became more than just an academic assignment because it involved complete end-to-end implementation starting from backend setup to frontend debugging and integration. It helped in building stronger understanding of real MERN stack development workflows and software project structuring.

The system can still be improved further in future by adding:

*** Authentication and Authorization**
*** Admin Dashboard**
*** Search and Sorting**
*** Book Reservation System**
*** Fine Management**
*** Cloud Storage**
*** Advanced Analytics**
*** Deployment Support**

In conclusion, the project was completed successfully and provided valuable practical experience in designing, developing, debugging, and integrating a modern Full Stack MERN application.